TASK 3 – DBMS

Seller and Inventory Management System

1. Aim

To develop a MySQL-based Seller and Inventory Management System for maintaining seller records, product information, stock levels, and product availability.

2. Objectives

- **Create and manage seller, product, and inventory tables.**
- **Store detailed information about sellers and their products.**
- **Connect products with their respective sellers.**
- **Record the quantity of products available in stock.**
- **Identify products that are in stock and out of stock.**
- **Perform insertion, modification, deletion, and retrieval operations.**
- **Generate useful inventory reports using SQL queries.**

3. Database Design

The database consists of three major tables:

Seller Table

- **Seller ID**
- **Seller Name**
- **Contact Number**
- **Email**

Product Table

- **Product ID**
- **Product Name**
- **Price**
- **Seller ID**

Inventory Table

- **Inventory ID**
- **Product ID**
- **Quantity**
- **Availability Status**

Relationship:

Seller → Product → Inventory

A seller can supply multiple products, while every product is associated with inventory information.

4. Create Database

```
CREATE DATABASE seller_inventory_db;

USE seller_inventory_db;
```

5. Create Seller Table

```
CREATE TABLE Seller (
  seller_id INT PRIMARY KEY,
  seller_name VARCHAR(50),
  phone VARCHAR(15),
  email VARCHAR(60)
);
```

6. Create Product Table

```
CREATE TABLE Product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(50),
  price DECIMAL(10,2),
  seller_id INT,
  FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)
);
```

7. Create Inventory Table

```
CREATE TABLE Inventory (
  inventory_id INT PRIMARY KEY,
  product_id INT,
  stock_quantity INT,
  status VARCHAR(20),
  FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

8. Insert Seller Records

```
INSERT INTO Seller VALUES
(1, 'Arun Electronics', '9845012345', 'arun@shop.com'),
(2, 'Meena Traders', '9789012345', 'meena@shop.com'),
(3, 'Sri Tech Mart', '9678012345', 'sritech@shop.com');
```

9. Insert Product Records

```
INSERT INTO Product VALUES
(201, 'Printer', 8500.00, 1),
(202, 'Web Camera', 1800.00, 2),
(203, 'Headset', 1500.00, 3),
(204, 'External Hard Disk', 6200.00, 1);
```

10. Insert Inventory Records

```
INSERT INTO Inventory VALUES
(11, 201, 12, 'Available'),
(12, 202, 8, 'Available'),
(13, 203, 0, 'Out of Stock'),
(14, 204, 6, 'Available');
```

11. Display Seller Records

```
SELECT * FROM Seller;
```

12. Display Product Records

```
SELECT * FROM Product;
```

13. Display Inventory Records

```
SELECT * FROM Inventory;
```

14. Display Seller-wise Product Details

```
SELECT
  Seller.seller_name,
  Product.product_name,
  Product.price
FROM Seller
INNER JOIN Product
ON Seller.seller_id = Product.seller_id;
```

15. Display Products Currently Available

```
SELECT
  Product.product_name,
  Inventory.stock_quantity
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.status = 'Available';
```

16. Display Out-of-Stock Products

```
SELECT
  Product.product_name,
```

```
  Inventory.stock_quantity
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.status = 'Out of Stock';
```

17. Generate Inventory Report

```
SELECT
  Product.product_id,
  Product.product_name,
  Inventory.stock_quantity,
  Inventory.status
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id;
```

18. Update Product Stock

For example, if the stock of the printer is increased:

```
UPDATE Inventory
SET stock_quantity = 20,
  status = 'Available'
WHERE product_id = 201;
```

19. Delete Product Record

Before deleting a product, its related inventory record can be removed:

```
DELETE FROM Inventory
WHERE product_id = 204;

DELETE FROM Product
WHERE product_id = 204;
```

20. Sample Output

Product	Stock Quantity	Status
Printer	12	Available
Web Camera	8	Available
Headset	0	Out of Stock
External Hard Disk	6	Available

21. Conclusion

The Seller and Inventory Management System provides an organized method for storing and managing seller, product, and inventory information. The use of primary and foreign keys establishes proper relationships between the tables. SQL operations such as insertion, updating, deletion, joining, and filtering make it possible to manage inventory efficiently. The system can also identify available and out-of-stock products and produce clear inventory reports.